Pushkar Gudla
Email: ai24btech11012@iith.ac.in
Course: AI3403 - Multi-Agent Systems (MAS)
Instructor: Dr. Venkatraman Renganathan

Assignment - 4
ID: AI24BTECH11012
Term: Fall 2026
Due Date: 9th October, 2026

Problem 1:
Data-Driven Epidemic Modelling and Vaccination on Networks

Consider a population of N individuals connected through an undirected contact network $G = (V, E)$, with adjacency matrix $A \in \mathbb{R}^{N \times N}$. Each individual can be in one of the following three states: Susceptible, Infected, and Recovered. The disease evolution is assumed to approximately follow an SIR model $S \longrightarrow I \longrightarrow R$. You are provided with two synthetic datasets:

- `network.csv`: the adjacency matrix, or equivalently the edge list, of the contact network;
- `epidemic_data.csv`: the observed infection state of every individual over a sequence of days.

The true transmission rate β and recovery rate γ used to generate the data are **not provided**. Your objective is to reconstruct the epidemic from the data, estimate the unknown SIR parameters, simulate the epidemic, determine whether the observed spreading behaviour corresponds to an epidemic, and design a vaccination strategy based on the topology of the contact network.

1. From the individual-level data, compute at every day k, the quantities $S[k]$, $I[k]$, and $R[k]$. Plot them on the same figure. From the data, report the following:
   (a) the maximum number of simultaneously infected individuals,
   (b) the day on which the infection reaches its maximum,
   (c) the final number of individuals who experienced infection,
   (d) the fraction of the population that experienced infection.

2. Consider the continuous-time SIR model. Assume that the sampling interval is $\Delta t = 1$ day. Using a forward-Euler approximation, we can discretize the recovery dynamics as

$$R[k+1] - R[k] \approx \gamma I[k] \Delta t$$

   When $I[k] \neq 0$, an estimate of the recovery rate can be obtained as

$$\gamma_k = \frac{R[k+1] - R[k]}{I[k]\Delta t}$$

   Use the above idea and the given data set to estimate γ. You may, compute the estimate as

$$\hat{\gamma} = \frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} \frac{R[k+1] - R[k]}{I[k]\Delta t}$$

   where $\mathcal{K} = \{k : I[k] > 0\}$ denotes the days when the number of infected people was strictly positive. Using estimated value $\hat{\gamma}$, report the approximate average infectious duration.

3. Use a similar approach to discretize the equation for susceptible population and estimate the transmission rate β whenever $S[k]I[k] \neq 0$ and inferring the $\mathcal{K}_\beta = \{k : S[k]I[k] > 0\}$ (from the given data set) which denotes the days when $S[k]I[k] > 0$. Report the estimated transmission rate $\hat{\beta}$. Using the estimates $\hat{\beta}$ and $\hat{\gamma}$, compute the reproduction number and interpret the obtained value as to whether the infection can initially grows or tends to die out. Also comment whether the observed disease has the potential to generate an epidemic.

4. Using parameters $\hat{\beta}$ and $\hat{\gamma}$, form the continuous time SIR model. Infer the initial values from the given data as $S(0) = S[0]$, $I(0) = I[0]$, $R(0) = R[0]$. Simulate the SIR model and plot the simulated trajectories of $S(t)$, $I(t)$, and $R(t)$. Comment on how well the fitted SIR model explains the synthetic epidemic data.

5. You should now use both the data and the mathematical model to determine whether the observed infection constituted an epidemic. Your answer must be based on evidences like the observed infection trajectory $I[k]$, the estimated reproduction number $\hat{R}_0$, the magnitude of the infection peak, and the final fraction of the population infected.

6. Suppose that a vaccine is available before the epidemic begins, but only a limited number of individuals can be vaccinated. Assume that you are permitted to vaccinate at most $B = 15$ individuals. A vaccinated individual is assumed to be completely immune and does not participate in disease transmission. Your task is to determine which nodes in the network should be vaccinated in order to reduce the spread of infection. Compare the following vaccination strategies:

   (a) **Random vaccination:** Select B nodes uniformly at random.

   (b) **Degree-based vaccination:** Compute the degree $d_i = \sum_{j=1}^{N} a_{ij}$ of every node and vaccinate the B nodes having the largest degree.

   (c) **Eigenvector-centrality-based vaccination:** Compute the dominant eigenvector v satisfying $Av = \lambda_{\max}(A)v$. Vaccinate the B nodes corresponding to the largest entries of v.

   For every vaccination strategy, remove the vaccinated individuals from the susceptible population before the epidemic starts (that is at time $t = 0$) and simulate the epidemic again. Compare the following quantities:

$$I_{\max} = \max_k I[k], \qquad T_{\text{peak}} = \arg\max_k I[k], \qquad N_{\text{infected}} = N - S[\infty]$$

   Plot the infection trajectories for all vaccination strategies. Answer the following questions.

   (a) Which vaccination strategy performed best?

   (b) Why did vaccinating certain nodes have a larger effect than vaccinating others?

   (c) Is vaccinating the highest-degree nodes always the same as vaccinating the nodes with highest eigenvector centrality?

Solution: The data has $N = 300$ individuals observed over 101 days, and the contact network has 1558 edges. All results below come from the attached Python code.

1. SIR counts from the data. At each day k, $S[k]$, $I[k]$ and $R[k]$ are the number of individuals in the respective state (so $S[k] + I[k] + R[k] = N$).

(a) Maximum number of simultaneously infected: $\max_k I[k] = 110$

(b) The peak occurs on day 34

(c) Total number of individuals ever infected: 282

(d) Fraction of the population infected: $282/300 = 0.940$

2. Recovery rate. Averaging $\gamma_k = \dfrac{R[k+1] - R[k]}{I[k]\Delta t}$ over $\mathcal{K} = \{k : I[k] > 0\}$ with $\Delta t = 1$ gives

$$\hat{\gamma} = 0.0713 \text{ day}^{-1}, \qquad \text{average infectious duration} = \frac{1}{\hat{\gamma}} \approx 14.0 \text{ days}$$

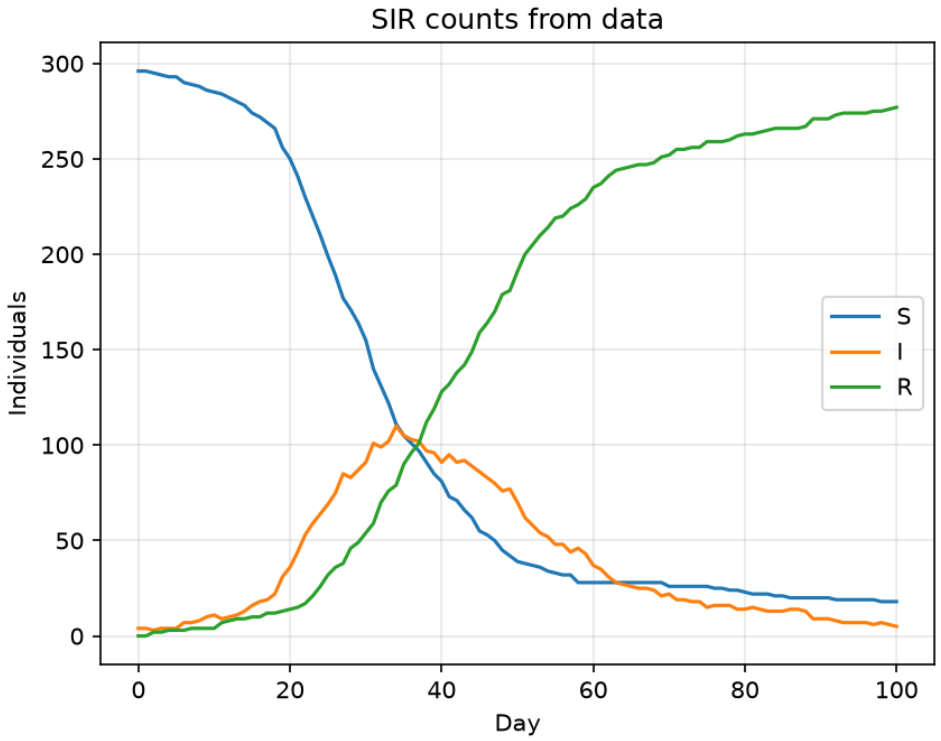


Figure 1: $S[k]$, $I[k]$, $R[k]$ computed from the individual-level data.

3. Transmission rate and reproduction number. With $s = S/N$ and $i = I/N$, the SIR equation is $\dot{s} = -\beta si$. The forward-Euler discretization gives

$$s[k+1] - s[k] \approx -\beta\, s[k]\, i[k]\, \Delta t \quad \Longrightarrow \quad \beta_k = \frac{s[k] - s[k+1]}{s[k]\, i[k]\, \Delta t}$$

Averaging over $\mathcal{K}_\beta = \{k : S[k]I[k] > 0\}$:

$$\hat{\beta} = 0.2391 \text{ day}^{-1}, \qquad \hat{R}_0 = \frac{\hat{\beta}}{\hat{\gamma}} = \frac{0.2391}{0.0713} \approx 3.35$$

Interpretation. Since $\hat{R}_0 > 1$, one infected individual infects on average about 3.35 others during the infectious period, so the infection *grows initially*. Near the start, $\dot{I} \approx (\hat{\beta} - \hat{\gamma})I$ with growth rate 0.168 per day (doubling time ≈ 4.1 days). The disease therefore has clear potential to generate an epidemic.

4. Simulation of the fitted SIR model. The continuous-time model

$$\dot{s} = -\hat{\beta}\, s\, i, \qquad \dot{i} = \hat{\beta}\, s\, i - \hat{\gamma}\, i, \qquad \dot{r} = \hat{\gamma}\, i$$

was integrated with the initial values $S(0) = S[0]$, $I(0) = I[0]$, $R(0) = R[0]$.

Comment. The model captures the overall shape well: the RMSE of $I(t)$ is 11.7 individuals, out of a peak of 110. The model peak is 103.6 on day 31.1, against 110 on day 34 in the data. The model final infected fraction is 0.959, against 0.940 in the data. The small mismatch is expected because the ODE is a well-mixed (mean-field) model, whereas the data was generated on a network in which each person only meets a limited set of neighbours.

5. Was it an epidemic? Yes. The evidence is:

- **Trajectory:** $I[k]$ rises from a handful of cases to a clear peak on day 34 before declining, which is the typical epidemic curve.

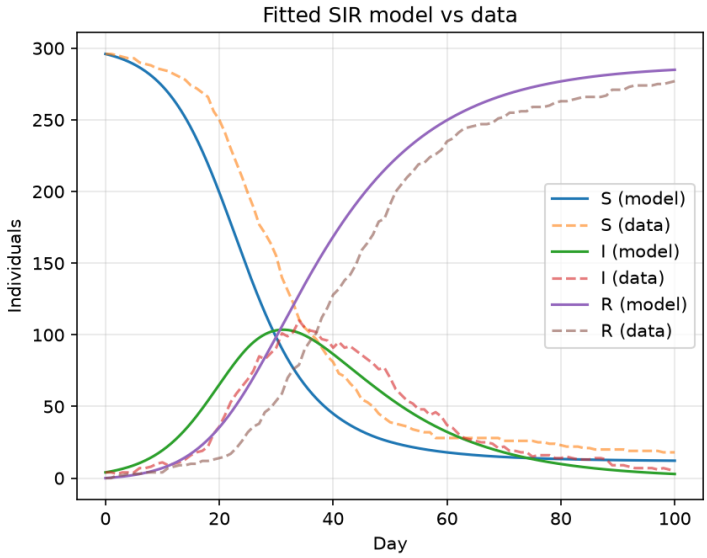


Figure 2: Fitted SIR model (solid) against the observed data (dashed).

- **Reproduction number:** $\hat{R}_0 \approx 3.35 > 1$, so the outbreak is self-sustaining. The herd-immunity threshold is $1 - 1/\hat{R}_0 \approx 70\%$.
- **Peak:** 110 people, i.e. 36.7% of the population, were infected simultaneously.
- **Final size:** 94% of the population was infected, well above the herd-immunity threshold, and this is consistent with the model's prediction of 95.9%.

6. Vaccination strategies. Vaccinated nodes are removed at $t = 0$ (initially infected nodes are not chosen for vaccination). The epidemic is simulated as a stochastic SIR process on the contact network, with per-edge infection rate $\hat{\beta}/\bar{d}$ ($\bar{d} = 10.4$ is the mean degree) and recovery rate $\hat{\gamma}$. Results are averaged over 200 runs per strategy (a new random selection in each run for random vaccination). Since the vaccinated nodes are immune, N_{infected} counts the individuals who actually became infected, i.e. N minus the never-infected individuals (those still susceptible at the end plus the B vaccinated nodes).

Strategy	$I_{\max}$	T_{peak} (days)	N_{infected}
No vaccination	97.3	35.2	271.6
Random	87.0	38.6	253.7
Degree	**75.6**	41.8	**242.6**
Eigenvector	79.5	42.0	249.5

Table 1: Average outcomes for $B = 15$ vaccinated nodes.

(a) **Best strategy: degree-based vaccination.** It gives the lowest peak ($I_{\max}$ falls by about 22% relative to no vaccination) and the fewest total infections (N_{infected} falls from 271.6 to 242.6). Eigenvector-based vaccination is a close second ($I_{\max} = 79.5$, $N_{\text{infected}} = 249.5$), and random vaccination helps the least. Degree and eigenvector vaccination both delay the peak by about 6.6–6.8 days (35.2 → 41.8 and 42.0 respectively), and the difference between the two in T_{peak} is negligible.

(b) Vaccinating high-degree nodes has a larger effect because a node's role in the spread is proportional to how many contacts it has. Hubs are infected early and pass the disease to many neighbours. Removing 15 hubs deletes far more edges from the transmission network than removing 15 typical nodes: the degree-based picks have mean degree 17.5, against a network mean of 10.4. Fewer edges lower the spectral radius $\lambda_{\max}$ of the remaining network, and hence the effective reproduction number

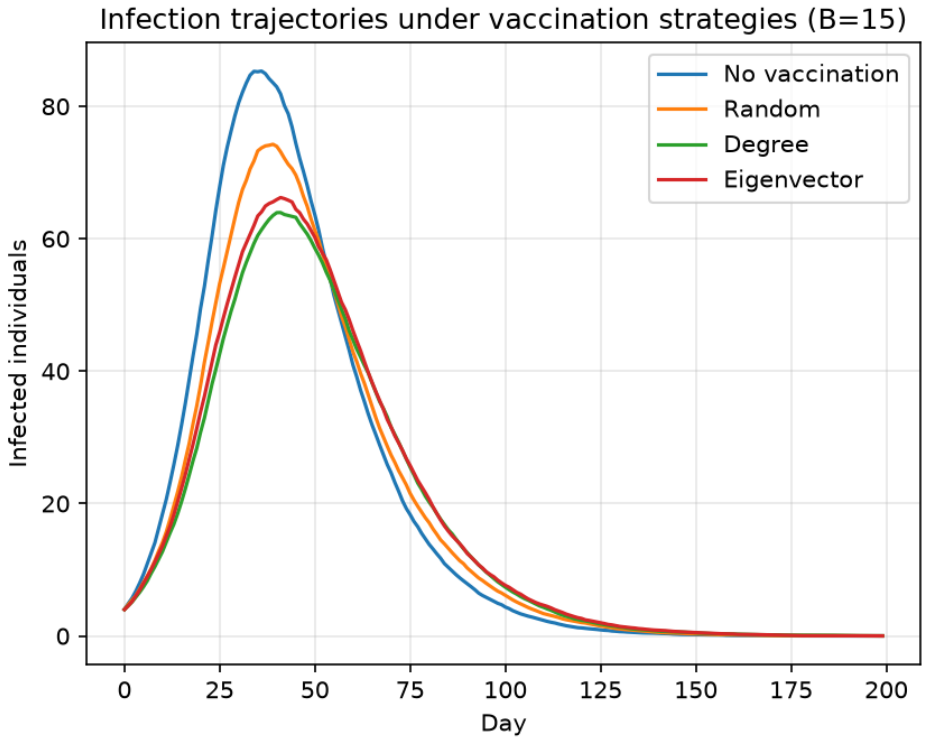


Figure 3: Mean infection trajectories for all vaccination strategies.

(which scales roughly like $\frac{\hat{\beta}}{d\hat{\gamma}}\lambda_{\max}$), so the outbreak grows more slowly and stays smaller. Random picks are mostly low-degree nodes, so they remove few edges.

(c) No, they are not always the same. Degree counts only the number of neighbours, $d_i = \sum_j a_{ij}$. Eigenvector centrality satisfies $v_i = \frac{1}{\lambda_{\max}} \sum_j a_{ij} v_j$, so it also rewards being connected to *important* nodes. A node with few but highly central neighbours can rank high in eigenvector centrality yet low in degree, and a node with many peripheral neighbours ranks high in degree but lower in eigenvector centrality. In this network only 10 of the 15 selected nodes are common to both rankings. The two rankings are guaranteed to coincide only in special cases such as regular graphs, where all degrees are equal and $v \propto \mathbf{1}$.

Code for Problem 1

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

NET_FILE = "network.csv"
DATA_FILE = "epidemic_data.csv"
STATE_MAP = {"S": 0, "I": 1, "R": 2}
DT = 1.0
B = 15
N_RUNS = 200
T_SIM = 200
rng = np.random.default_rng(0)


# ---------- Load data ----------
df = pd.read_csv(DATA_FILE)
```

```
df["state"] = df["state"].str.strip().str.upper().map(STATE_MAP)
wide = df.pivot(index="day", columns="node", values="state").sort_index()
states = wide.values.astype(int)
node_ids = list(wide.columns)
node_index = {n: idx for idx, n in enumerate(node_ids)}
N = len(node_ids)
T = states.shape[0]

edges = pd.read_csv(NET_FILE)
A = np.zeros((N, N))
for u, w in zip(edges["source"], edges["target"]):
    if u in node_index and w in node_index and u != w:
        A[node_index[u], node_index[w]] = 1
A = np.maximum(A, A.T)

print("Loaded: %d days, %d individuals, %d edges" % (T, N, int(A.sum() / 2)))


# ---------- Q1: S, I, R over time ----------
S = (states == 0).sum(axis=1).astype(float)
I = (states == 1).sum(axis=1).astype(float)
R = (states == 2).sum(axis=1).astype(float)
days = np.arange(T)

plt.figure()
plt.plot(days, S, label="S")
plt.plot(days, I, label="I")
plt.plot(days, R, label="R")
plt.xlabel("Day")
plt.ylabel("Individuals")
plt.title("SIR counts from data")
plt.legend()
plt.grid(alpha=0.3)
plt.savefig("q1_sir_data.png", dpi=150)

ever_infected = ((states == 1) | (states == 2)).any(axis=0).sum()
print("\nQ1(a) max infected:", int(I.max()))
print("Q1(b) day of peak:", int(I.argmax()))
print("Q1(c) total ever infected:", int(ever_infected))
print("Q1(d) fraction infected: %.3f" % (ever_infected / N))


# ---------- Q2: gamma ----------
K = [k for k in range(T - 1) if I[k] > 0]
gamma_hat = np.mean([(R[k + 1] - R[k]) / (I[k] * DT) for k in K])
print("\nQ2 gamma_hat = %.4f per day" % gamma_hat)
print("Q2 avg infectious duration = %.2f days" % (1 / gamma_hat))


# ---------- Q3: beta and R0 ----------
s, i = S / N, I / N
Kb = [k for k in range(T - 1) if S[k] * I[k] > 0]
beta_hat = np.mean([(s[k] - s[k + 1]) / (s[k] * i[k] * DT) for k in Kb])
R0 = beta_hat / gamma_hat
print("\nQ3 beta_hat = %.4f per day" % beta_hat)
print("Q3 R0 = %.3f" % R0)
print("Q3 infection initially", "grows" if R0 > 1 else "dies out")


# ---------- Q4: simulate continuous SIR ----------
```

```
def sir_ode(t, y):
    S_, I_, R_ = y
    return [-beta_hat * S_ * I_, beta_hat * S_ * I_ - gamma_hat * I_, gamma_hat
       * I_]


t_eval = np.linspace(0, T - 1, 400)
sol = solve_ivp(sir_ode, (0, T - 1), [s[0], i[0], R[0] / N], t_eval=t_eval)

plt.figure()
for idx, (name, data) in enumerate(zip("SIR", (S, I, R))):
    plt.plot(sol.t, sol.y[idx] * N, label=name + " (model)")
    plt.plot(days, data, "--", alpha=0.6, label=name + " (data)")
plt.xlabel("Day")
plt.ylabel("Individuals")
plt.title("Fitted SIR model vs data")
plt.legend()
plt.grid(alpha=0.3)
plt.savefig("q4_sir_model.png", dpi=150)

model_I = np.interp(days, sol.t, sol.y[1] * N)
rmse = np.sqrt(np.mean((model_I - I) ** 2))
print("\nQ4 RMSE of I(t): %.2f individuals" % rmse)
print("Q4 model peak: %.1f on day %.1f" % (sol.y[1].max() * N, sol.t[sol.y[1].
   argmax()]))
print("Q4 model final infected fraction: %.3f" % (1 - sol.y[0, -1]))


# ---------- Q5: epidemic evidence ----------
print("\nQ5 evidence")
print("  R0 = %.3f" % R0)
print("  peak infected = %d (%.1f%% of population)" % (I.max(), 100 * I.max() /
   N))
print("  final fraction infected = %.3f" % (ever_infected / N))
print("  epidemic:", "YES" if R0 > 1 and I.max() > I[0] else "NO")


# ---------- Q6: vaccination on the network ----------
deg = A.sum(axis=1)
beta_edge = beta_hat / deg.mean()
p_inf = 1 - np.exp(-beta_edge * DT)
p_rec = 1 - np.exp(-gamma_hat * DT)
init_infected = np.where(states[0] == 1)[0]


def simulate(vaccinated):
    st = np.zeros(N, dtype=int)
    st[vaccinated] = 3
    st[init_infected] = 1
    curve = np.zeros(T_SIM)
    for t in range(T_SIM):
        infected = (st == 1)
        curve[t] = infected.sum()
        if curve[t] == 0:
            break
        n_inf_nbrs = A @ infected
        new_inf = (st == 0) & (rng.random(N) < 1 - (1 - p_inf) ** n_inf_nbrs)
        new_rec = infected & (rng.random(N) < p_rec)
        st[new_inf] = 1
        st[new_rec] = 2
```

```
    total_infected = (st == 1).sum() + (st == 2).sum()
    return curve, total_infected


candidates = np.setdiff1d(np.arange(N), init_infected)


def pick_random():
    return rng.choice(candidates, B, replace=False)


def pick_degree():
    return candidates[np.argsort(-deg[candidates])[:B]]


eigvals, eigvecs = np.linalg.eigh(A)
v = np.abs(eigvecs[:, -1])


def pick_eigen():
    return candidates[np.argsort(-v[candidates])[:B]]


strategies = {
    "No vaccination": lambda: np.array([], dtype=int),
    "Random": pick_random,
    "Degree": pick_degree,
    "Eigenvector": pick_eigen,
}

results = {}
plt.figure()
print("\nQ6 results (mean over %d runs)" % N_RUNS)
print("%-16s %8s %8s %10s" % ("Strategy", "Imax", "Tpeak", "Ninfected"))
for name, pick in strategies.items():
    curves, totals = [], []
    for _ in range(N_RUNS):
        c, tot = simulate(pick())
        curves.append(c)
        totals.append(tot)
    mean_curve = np.mean(curves, axis=0)
    Imax = np.mean([c.max() for c in curves])
    Tpeak = np.mean([c.argmax() for c in curves])
    Ninf = np.mean(totals)
    results[name] = (Imax, Tpeak, Ninf)
    print("%-16s %8.1f %8.1f %10.1f" % (name, Imax, Tpeak, Ninf))
    plt.plot(mean_curve, label=name)

plt.xlabel("Day")
plt.ylabel("Infected individuals")
plt.title("Infection trajectories under vaccination strategies (B=%d)" % B)
plt.legend()
plt.grid(alpha=0.3)
plt.savefig("q6_vaccination.png", dpi=150)

best = min((k for k in results if k != "No vaccination"), key=lambda k: results[
    k][2])
print("\nQ6(a) best strategy:", best)

top_deg = set(pick_degree())
```

```
top_eig = set(pick_eigen())
print("Q6(c) overlap between degree and eigenvector picks: %d of %d" % (len(
    top_deg & top_eig), B))
print("      degree of degree-picks: mean %.1f | degree of eigenvector-picks:
    mean %.1f"
      % (deg[list(top_deg)].mean(), deg[list(top_eig)].mean()))
print("      network mean degree: %.1f, lambda_max: %.2f" % (deg.mean(), eigvals
    [-1]))

plt.show()
```

Problem 2

Temporal Logic Specification for Cyber-security System

Consider an online authentication system for a secure computing service. Each user account can experience login attempts, failed authentications, successful authentications, account locking, administrator verification, password-reset requests, user notifications, and account unlocking. The security system also continuously monitors quantities such as the user's login anomaly score, the rate of failed login attempts, and the confidence that the current user is legitimate. Your task is to express the security requirements given below using LTL, MTL, and STL. Simply writing a temporal-logic formula without explaining its meaning will not receive full credit.

Consider using the following Boolean atomic propositions:

`LoginFailure` : the current login attempt has failed,

`LoginSuccess` : the current login attempt is successful,

`AccountLocked` : the account is locked,

`AccountUnlocked` : the account is unlocked,

`AdminVerified` : administrator verification has been completed,

`PasswordResetRequested` : a password-reset request has been made,

`UserNotified` : the user has been notified of a security event,

`SuspiciousActivityDetected` : suspicious login behaviour has been detected.

Assume that the authentication system continuously monitors the following real-valued signals:

`AnomalyScore`(t) : login anomaly score at time t,

`FailedAttemptRate`(t) : number/rate of failed login attempts over a recent time window,

`LegitimateUserConfidence`(t) : confidence that the current user is legitimate,

`AccountLockStatus`(t) : account-lock status at time t.

Assume that

$$\texttt{AccountLockStatus}(t) = \begin{cases} 1, & \text{the account is locked,} \\ 0, & \text{the account is unlocked.} \end{cases}$$

Larger values of `AnomalyScore`(t) and `FailedAttemptRate`(t) indicate more suspicious behaviour, whereas larger values of `LegitimateUserConfidence`(t) indicate greater confidence that the user is legitimate. Using LTL, encode the following security requirements. For each specification, write the corresponding LTL formula and briefly explain why your formula represents the given requirement.

A1. Whenever an account is locked, a successful login must not occur.

A2. Once the account becomes locked, it should remain locked until administrator verification has been completed.

A3. Whenever administrator verification is completed for a locked account, the account must eventually become unlocked.

A4. Whenever a password-reset request is received, the account must eventually either become unlocked or administrator verification must be completed.

A5. A successful login and a failed login attempt must never occur simultaneously.

A6. Whenever an account becomes locked, the corresponding user must eventually be notified of the security event.

Encode each of the following requirements using MTL. For each requirement, write the corresponding MTL formula; and explain briefly how the time interval appearing in the formula represents the stated timing requirement.

B1. Whenever `SuspiciousActivityDetected` becomes true, the account must become locked within 10 seconds.

B2. Whenever the account becomes locked, the user must be notified within 30 seconds.

B3. Whenever a password-reset request is made, administrator verification must be completed within 10 minutes.

B4. Once an account becomes locked, it must remain locked for at least 60 seconds unless administrator verification has already been completed.

B5. Once administrator verification has been successfully completed, the account must become unlocked within 2 minutes.

B6. Whenever a failed-login event occurs, then within the following 20 seconds at least one of the following must happen: 1) a legitimate login succeeds, or 2) the account becomes locked.

Suppose that the security system now uses continuous-valued monitoring signals. Assume the following security thresholds $A_{\text{crit}} = 0.8$, $F_{\text{crit}} = 5$, $C_{\text{safe}} = 0.9$. Encode the following requirements using STL. For each STL requirement, identify the atomic signal predicates, write the corresponding STL formula, and explain the physical meaning of the formula.

C1. During normal operation, the login anomaly score should always remain below the critical value meaning that $\texttt{AnomalyScore}(t) < A_{\text{crit}}$. Express this requirement using STL.

C2. Whenever $\texttt{AnomalyScore}(t) > A_{\text{crit}}$, the account must become locked within 10 seconds.

C3. Whenever $\texttt{FailedAttemptRate}(t) > F_{\text{crit}}$, the account must become locked within 5 seconds.

C4. Suppose an account is currently locked. Whenever both $\texttt{LegitimateUserConfidence}(t) > C_{\text{safe}}$ and $\texttt{AnomalyScore}(t) < 0.3$ hold, the account should become unlocked within 2 minutes.

C5. Whenever $\texttt{AnomalyScore}(t) > 0.8$ and $\texttt{LegitimateUserConfidence}(t) < 0.5$, the account must remain locked.

C6. Suppose that at some instant, $\texttt{AnomalyScore}(t) > 0.8$ and continues to remain above 0.8 for at least 5 seconds. Express the requirement that such persistent suspicious behaviour must cause the account to become locked within the following 5 seconds.

Answer the following conceptual questions.

D1. Why is MTL more appropriate than LTL for a requirement such as
"The account must be locked within 10 seconds after suspicious activity is detected."

D2. Why is STL more appropriate when security requirements involve real-valued signals such as $\texttt{AnomalyScore}(t)$, $\texttt{FailedAttemptRate}(t)$, $\texttt{LegitimateUserConfidence}(t)$?

D3. Explain the conceptual difference between the following three specifications:

$$\square\big(\texttt{SuspiciousActivityDetected} \rightarrow \Diamond\, \texttt{AccountLocked}\big)$$

$$\square\big(\texttt{SuspiciousActivityDetected} \rightarrow \Diamond_{[0,10]}\, \texttt{AccountLocked}\big)$$

and

$$\square\Big(\big(\texttt{AnomalyScore}(t) > 0.8\big) \rightarrow \Diamond_{[0,10]}\big(\texttt{AccountLockStatus}(t) = 1\big)\Big)$$

Clearly identify which of these formulas belongs to LTL, MTL, and STL, and explain what additional expressive capability is introduced when moving from one logic to the next.

Construct examples satisfying the following requirements.

E1. Construct one Boolean event trace that satisfies all the LTL specifications in Part A.

E2. Construct one Boolean event trace that violates at least two of the LTL specifications in Part A. For each violation, identify the specification and explain why the trace violates it.

E3. Construct one timed event trace that satisfies all the MTL specifications in Part B. The trace should explicitly indicate the times at which events occur.

E4. Construct one timed trace in which `SuspiciousActivityDetected` occurs, but the requirement "the account must become locked within 10 seconds" is violated. Clearly indicate:

- the time at which suspicious activity is detected;
- the time at which the account becomes locked, if it does;
- the MTL specification that is violated.

Solution:

Conventions. $\square$ = always, $\Diamond$ = eventually, $\mathcal{U}$ = until. MTL adds a time interval to the operators, e.g. $\Diamond_{[a,b]}\varphi$ means "φ holds at some time in $[t+a,\, t+b]$". STL replaces Boolean propositions by predicates on real-valued signals of the form $g(x(t)) > 0$. All times are in seconds (2 min = 120 s, 10 min = 600 s). In Part B, `AccountLocked`, `AccountUnlocked` and `AdminVerified` are *states* that stay true once they switch on (in particular, `AdminVerified` stays true after verification), while the other propositions are events true only at the instant they occur. "Whenever p becomes true" is written $\uparrow p$, which holds exactly at the instant p switches from false to true (for an event, its occurrence).

Part A: LTL

A1. $\square\big(\texttt{AccountLocked} \rightarrow \neg\texttt{LoginSuccess}\big)$
At every instant, if the account is locked then no successful login occurs.

A2. $\square\big(\texttt{AccountLocked} \rightarrow (\texttt{AccountLocked}\,\mathcal{U}\,\texttt{AdminVerified})\big)$
Whenever the account is locked, it stays locked until administrator verification holds. (Since $\mathcal{U}$ is a strong until, verification must eventually happen. If the account were allowed to stay locked forever without verification, replace $\mathcal{U}$ by the weak until $\mathcal{W}$.)

A3. $\square\big((\texttt{AccountLocked} \wedge \texttt{AdminVerified}) \rightarrow \Diamond\, \texttt{AccountUnlocked}\big)$
If verification is completed while the account is locked, the account is eventually unlocked.

A4. $\square\big(\texttt{PasswordResetRequested} \rightarrow \Diamond(\texttt{AccountUnlocked} \vee \texttt{AdminVerified})\big)$
Every reset request is eventually followed by an unlock or by completed verification (at least one of the two).

A5. $\square\, \neg(\texttt{LoginSuccess} \wedge \texttt{LoginFailure})$
At no instant are both true, i.e. they are mutually exclusive.

A6. $\square\big(\texttt{AccountLocked} \rightarrow \Diamond\, \texttt{UserNotified}\big)$
Whenever the account is locked, the user is notified at that instant or some time later.

Part B: MTL

B1. $\square\big(\texttt{SuspiciousActivityDetected} \rightarrow \Diamond_{[0,10]}\, \texttt{AccountLocked}\big)$
The interval $[0, 10]$ means the lock must occur between 0 and 10 seconds after the detection.

B2. $\square\big(\uparrow\texttt{AccountLocked} \rightarrow \Diamond_{[0,30]}\, \texttt{UserNotified}\big)$
$[0, 30]$: the notification must come within 30 seconds of the instant the account becomes locked.

B3. $\square\big(\texttt{PasswordResetRequested} \rightarrow \Diamond_{[0,600]}\, \texttt{AdminVerified}\big)$
$[0, 600]$: verification is completed within 10 minutes (600 s) of the request.

B4. $\square\big(\uparrow\texttt{AccountLocked} \rightarrow \square_{[0,60]}(\texttt{AccountLocked} \vee \texttt{AdminVerified})\big)$
From the instant the account becomes locked, throughout the next 60 seconds ($[0, 60]$) the account is still locked *or* verification has already been completed. So an early unlock within 60 seconds is only allowed after verification.

B5. $\square\big(\uparrow\texttt{AdminVerified} \rightarrow \Diamond_{[0,120]}\, \texttt{AccountUnlocked}\big)$
$[0, 120]$: the account is unlocked within 2 minutes of the instant verification is completed.

B6. $\square\big(\texttt{LoginFailure} \rightarrow \Diamond_{[0,20]}(\texttt{LoginSuccess} \vee \texttt{AccountLocked})\big)$
After each failed login, within 20 seconds either a successful login occurs or the account gets locked (the disjunction gives the "at least one" condition).

Part C: STL

Let $a(t) = \texttt{AnomalyScore}(t)$, $f(t) = \texttt{FailedAttemptRate}(t)$, $c(t) = \texttt{LegitimateUserConfidence}(t)$ and $\ell(t) = \texttt{AccountLockStatus}(t)$, with $A_{\text{crit}} = 0.8$, $F_{\text{crit}} = 5$, $C_{\text{safe}} = 0.9$. The atomic predicates (each of the form $\mu(x(t)) > 0$) are:

Predicate	**Meaning**
$\mu_1 := A_{\text{crit}} - a(t) > 0$	anomaly score below critical
$\mu_2 := a(t) - A_{\text{crit}} > 0$	anomaly score above critical (0.8)
$\mu_3 := f(t) - F_{\text{crit}} > 0$	failed-attempt rate above 5
$\mu_4 := c(t) - C_{\text{safe}} > 0$	confidence above 0.9
$\mu_5 := 0.3 - a(t) > 0$	anomaly score below 0.3
$\mu_6 := 0.5 - c(t) > 0$	confidence below 0.5
$\mu_L := \ell(t) - 0.5 > 0$	account locked ($\ell = 1$)
$\mu_U := 0.5 - \ell(t) > 0$	account unlocked ($\ell = 0$)

Since $\ell(t) \in \{0, 1\}$, the equality $\ell(t) = 1$ is written as the predicate μ_L.

C1. $\square\, \mu_1 \;=\; \square\big(a(t) < A_{\text{crit}}\big)$
At all times the anomaly score stays strictly below 0.8.

C2. $\square\big(\mu_2 \rightarrow \Diamond_{[0,10]}\, \mu_L\big)$
Whenever the anomaly score exceeds 0.8, the lock status becomes 1 within 10 seconds.

C3. $\square\big(\mu_3 \rightarrow \Diamond_{[0,5]}\, \mu_L\big)$
Whenever the failed-attempt rate exceeds 5, the account is locked within 5 seconds.

C4. $\square\big((\mu_L \wedge \mu_4 \wedge \mu_5) \rightarrow \Diamond_{[0,120]}\, \mu_U\big)$
If the account is locked while the user is very likely legitimate ($c > 0.9$) and the anomaly is low ($a < 0.3$), the account becomes unlocked within 2 minutes.

C5. $\square\big((\mu_2 \wedge \mu_6) \rightarrow \mu_L\big)$
At every instant where the anomaly score is above 0.8 and the confidence is below 0.5, the account is locked. So it stays locked for as long as this condition persists.

C6. $\square\big(\square_{[0,5]}\, \mu_2 \rightarrow \Diamond_{[5,10]}\, \mu_L\big)$
If the anomaly score stays above 0.8 during the whole window $[t, t+5]$ (persistent suspicious behaviour), the account must be locked at some time in $[t+5, t+10]$, i.e. within 5 seconds after the persistence has been established.

Part D: Conceptual Questions

D1. LTL only talks about the *order* of events: $\Diamond$ `AccountLocked` is satisfied even if the lock happens an hour after the detection. It has no way to measure elapsed time. MTL attaches a real-time interval to the temporal operators, so $\Diamond_{[0,10]}$ `AccountLocked` says that the lock occurs at most 10 seconds after the detection, which is exactly the requirement.

D2. LTL and MTL work on Boolean propositions, so a condition like "the anomaly score is high" has to be abstracted into a proposition beforehand, and the threshold is hidden outside the formula. STL puts the thresholds directly in the formula through predicates such as $a(t) > 0.8$ evaluated on continuous-time signals. STL also has *robustness semantics*, which measure by how much a signal satisfies or violates a requirement (e.g. how far $a(t)$ is below 0.8). This is useful for monitoring and for tuning thresholds.

D3.

- $\square$(`SuspiciousActivityDetected` $\rightarrow \Diamond$ `AccountLocked`) is an **LTL** formula: after every detection the account is eventually locked, with no bound on the delay.
- $\square$(`SuspiciousActivityDetected` $\rightarrow \Diamond_{[0,10]}$ `AccountLocked`) is an **MTL** formula: same structure, but the lock must occur within 10 seconds. *New capability: quantitative time bounds.*
- $\square\big((a(t) > 0.8) \rightarrow \Diamond_{[0,10]}(\ell(t) = 1)\big)$ is an **STL** formula: the trigger is no longer a Boolean event but a threshold on a real-valued signal, and the response is a condition on the lock-status signal. *New capability: predicates on real-valued signals (and robustness).*

Each logic contains the previous one: LTL is MTL with the interval $[0, \infty)$, and MTL is STL applied to $\{0, 1\}$-valued signals.

Part E: Example Traces

E1: Boolean trace satisfying all of A1–A6. The last state s_7 repeats forever. Each step lists the propositions that are true.

Step	True propositions
s_0	`AccountUnlocked`, `LoginSuccess`
s_1	`AccountUnlocked`, `LoginFailure`
s_2	`AccountUnlocked`, `LoginFailure`, `SuspiciousActivityDetected`
s_3	`AccountLocked`
s_4	`AccountLocked`, `PasswordResetRequested`
s_5	`AccountLocked`, `AdminVerified`, `UserNotified`
s_6	`AccountUnlocked`, `LoginSuccess`
s_7^ω	`AccountUnlocked`

Check. A1: `LoginSuccess` only occurs in s_0, s_6, where the account is unlocked. A2: the locked states s_3, s_4, s_5 are followed by `AdminVerified` at s_5 with the account locked until then. A3: verification at s_5 is followed by unlocking at s_6. A4: the reset at s_4 is followed by `AdminVerified` at s_5. A5: success and failure never share a step. A6: `UserNotified` holds at s_5, which is at or after every locked step.

E2: Boolean trace violating several specifications.

Step	True propositions
u_0	`AccountUnlocked`, `LoginFailure`
u_1	`AccountLocked`
u_2	`AccountLocked`, `LoginSuccess`, `LoginFailure`
u_3	`AccountUnlocked`
u_4^ω	`AccountUnlocked`

- **A1 violated:** at u_2 the account is locked and `LoginSuccess` occurs.
- **A5 violated:** at u_2, `LoginSuccess` and `LoginFailure` hold together.
- **A2 violated:** the account is locked at u_1 but is unlocked at u_3 without `AdminVerified` ever being true, so `AccountLocked` $\mathcal{U}$ `AdminVerified` fails.
- **A6 violated:** the account is locked at u_1 but `UserNotified` never occurs.

(A3 and A4 hold vacuously, since verification and reset requests never occur.)

E3: Timed trace satisfying all of B1–B6.

Time (s)	Event
0	`LoginFailure`
5	`SuspiciousActivityDetected`
8	`AccountLocked` (account stays locked over $[8, 260)$)
20	`UserNotified`
30	`PasswordResetRequested`
200	`AdminVerified` (stays true from now on)
260	`AccountUnlocked`

Check. B1: lock at 8, which is $3 \leq 10$ s after the detection at 5. B2: notification at 20, which is $12 \leq 30$ s after the lock at 8. B3: verification at 200, which is $170 \leq 600$ s after the request at 30. B4: after the lock at 8 the account is locked throughout $[8, 68]$ (in fact until 260), so the 60-second window is covered. B5: unlock at 260, which is $60 \leq 120$ s after verification at 200. B6: the failure at 0 is followed by the lock at $8 \leq 20$.

E4: Timed trace violating the 10-second requirement.

Time (s)	Event
12	`SuspiciousActivityDetected`
30	`AccountLocked`

The account is unlocked before $t = 30$.

- Suspicious activity is detected at $t = 12$ s.
- The account becomes locked at $t = 30$ s, which is 18 s later and therefore outside the allowed window $[12, 22]$.
- The violated specification is **B1**: $\square(\texttt{SuspiciousActivityDetected} \rightarrow \Diamond_{[0,10]}\, \texttt{AccountLocked})$. (If the account were never locked, the formula would be violated as well.)

Note that the untimed LTL version $\square(\texttt{SuspiciousActivityDetected} \rightarrow \Diamond\, \texttt{AccountLocked})$ is *satisfied* by this trace, which is exactly why MTL is needed (D1).

AI Disclosure: AI tools were used solely to assist with LaTeX formatting, organization, and improving the readability and presentation of this assignment. The content, analysis, and final answers were reviewed and verified by me.